

图像压缩中的线性代数方法研究

学号：XXXXXXXX 姓名：还好

摘要：图像压缩是数字图像处理中重要的研究方向，其核心问题在于如何在尽量减少信息损失的前提下降低数据冗余。线性代数为图像压缩提供了坚实的理论基础。本文从线性代数角度出发，介绍了图像压缩中常见的数学方法，并重点分析了快速傅里叶变换在图像压缩中的线性代数原理。

关键词：图像压缩；线性代数；快速傅里叶变换；基变换

1 引言

2 研究背景

随着数字图像与多媒体技术的迅速发展，图像数据规模不断增长。如何在保证图像质量的前提下降低存储与传输成本，成为图像处理领域的重要问题。图像压缩技术正是在这一背景下发展起来的，而其中大量方法均可归结为线性代数中的矩阵运算与线性变换。

3 快速傅里叶变换（FFT）

快速傅里叶变换（Fast Fourier Transform, FFT）是离散傅里叶变换（Discrete Fourier Transform, DFT）的一种高效计算方法。尽管 FFT 常被视为算法层面的优化，但从线性代数的角度来看，其本质是对离散傅里叶线性变换所对应矩阵结构的分解与重排。这一思想在图像压缩等应用中具有重要的理论意义。

3.1 傅立叶变换

傅里叶变换（Fourier Transform, FT）是一种分析信号的方法，它可以分析信号的成分，也可反过来用这些成分合成信号。通过使用天然具有正交性的三角函数（sin、cos）作为信号的成分，借助幅角的形式表示频率和相位，傅立叶变换实现了信号从时域到频域的变换。

设 $f(t)$ 是关于时间 t 的函数，则傅立叶变换可以检测频率 ω 的周期在 $f(t)$ 出现的程度：

$$F(\omega) = \mathbb{F}[f(t)] = \int_{-\infty}^{\infty} f(t)e^{-i\omega t} dt$$

它的逆变换是

$$f(t) = \mathbb{F}^{-1}[F(\omega)] = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega) e^{i\omega t} d\omega$$

逆变换的形式与正变换非常类似，分母 2π 恰好是指数函数的周期。

傅里叶变换相当于将时域的函数与周期为 2π 的复指数函数进行连续的内积。逆变换仍旧为一个内积。

3.2 离散傅里叶变换

离散傅里叶变换 (Discrete Fourier transform, DFT) 是傅里叶变换在时域和频域上都呈离散的形式，将信号的时域采样变换为其 DTFT (discrete-time Fourier transform) 的频域采样。

相应地，我们可以将 FT 积分形式的连续的函数内积改写成 DFT 求和形式的内积。

设 $\{x_n\}_{n=0}^{N-1}$ 是某一满足有限性条件的序列，它的离散傅里叶变换 (DFT) 为：

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i\frac{2\pi}{N}kn}$$

通常以符号 $\mathcal{F}$ 表示这一变换，即

$$\hat{x} = \mathcal{F}x$$

类似于积分形式，它的逆离散傅里叶变换 (IDFT) 为：

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{i\frac{2\pi}{N}kn}$$

可以记为：

$$x = \mathcal{F}^{-1}\hat{x}$$

实际上，DFT 和 IDFT 变换式前面的归一化系数并不重要。在上面的定义中，DFT 和 IDFT 前的系数分别为 1 和 $\frac{1}{N}$ 。有时我们会将这两个系数都改 $\frac{1}{\sqrt{N}}$ 。

离散傅里叶变换仍旧是时域到频域的变换。由于求和形式的特殊性，可以有其他的解释方法。

如果把序列 x_n 看作多项式 $f(x)$ 的 x^n 项系数，则计算得到的 X_k 恰好是多项式 $f(x)$ 代入单位根 $e^{-\frac{2\pi i k}{N}}$ 的点值 $f(e^{-\frac{2\pi i k}{N}})$ 。这便构成了卷积定理的另一种解释办法，即对多项式进行特殊的求值操作。离散傅里叶变换恰好是多项式在单位根处进行求值。

3.3 矩阵表示

由于离散傅立叶变换是一个线性算子，所以它可以用矩阵乘法来描述。在矩阵表示法中，离散傅立叶变换表示如下：

$$\begin{bmatrix} X_0 \\ X_1 \\ X_2 \\ \vdots \\ X_{N-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)(N-1)} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{N-1} \end{bmatrix}$$

其中 $\omega = e^{-i\frac{2\pi}{N}}$ 表示 N 次单位根。

设一维离散信号（或图像的一行像素）表示为向量：

$$\mathbf{x} = (x_0, x_1, \dots, x_{N-1})^T \in \mathbb{C}^N.$$

变换后频域强度表示为向量：

$$\mathbf{X} = (X_0, X_1, \dots, X_{N-1})^T \in \mathbb{C}^N.$$

离散傅里叶变换可以表示为如下线性变换：

$$\mathbf{X} = F_N \mathbf{x},$$

其中 $F_N \in \mathbb{C}^{N \times N}$ 为离散傅里叶变换矩阵

由此可见，DFT 本质上是一个复向量空间上的线性映射，其作用是将信号从标准基下的表示转换为一组由复指数函数张成的频率基下的表示。在图像压缩中，该过程对应于从空间域到频率域的变换。

3.4 多项式视角与基变换解释

若将向量 $\mathbf{x}$ 视为多项式

$$f(t) = \sum_{n=0}^{N-1} x_n t^n,$$

则 DFT 的结果满足

$$X_k = f(\omega_N^k), \quad k = 0, 1, \dots, N-1.$$

从线性代数角度看，该过程等价于一次坐标表示的改变，即从幂函数基下的系数表示，转换为复指数函数基下坐标的表示。这种基变换通常能够使信号的能量集中于少数低频分量，为图像压缩中的系数截断提供理论依据。

3.5 FFT 的矩阵分解思想

直接计算离散傅里叶变换需要进行一次 $N \times N$ 的矩阵-向量乘法，其时间复杂度为 $O(N^2)$ 。FFT 的核心思想在于，DFT 矩阵具有高度结构化的形式，可以利用分治的思想

通过置换矩阵与分块矩阵分解为多个低维 DFT 矩阵的组合，将复杂度降到 $O(N \log N)$ 。进一步还可以使用蝶形运算、位逆序置换等技巧加速运算。

这一过程本质上是将一个高维线性算子分解为若干低维线性算子的复合，体现了线性代数中通过矩阵分解与基重排来降低计算复杂度的基本思想。

3.6 FFT 的矩阵分解与分治解释

离散傅里叶变换可以表示为矩阵形式

$$\mathbf{X} = F_N \mathbf{x},$$

其中 F_N 为离散傅里叶变换矩阵。当 N 为 2 的幂时（事实上当 N 不为 2 的幂，我们可以通过补零的方式构造成 2 的幂）， F_N 具有良好的递归结构。

通过对输入向量进行适当的重排（对应于置换矩阵）， F_N 可以分解为如下形式：

$$F_N = \begin{pmatrix} I & D \\ I & -D \end{pmatrix} \begin{pmatrix} F_{N/2} & 0 \\ 0 & F_{N/2} \end{pmatrix} P,$$

其中 P 为置换矩阵， D 为由单位复根构成的对角矩阵， $F_{N/2}$ 为低维离散傅里叶变换矩阵。

该分解表明，FFT 的分治过程在本质上是对离散傅里叶变换矩阵的递归分块分解。通过将一个高维线性算子表示为多个低维线性算子与简单矩阵的乘积，FFT 显著降低了计算复杂度。

3.7 蝶形运算的线性代数意义

在快速傅里叶变换的分治过程中，所谓的“蝶形运算”并非单纯的计算技巧，而是局部线性变换在低维子空间中的体现。设在 FFT 的某一层分治中，已得到两个子问题的结果 u 与 v ，则其合并过程可表示为

$$\begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 1 & \omega \\ 1 & -\omega \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix},$$

其中 ω 为相应的单位复根。

从线性代数角度看，上式表明蝶形运算实质上是一个作用在二维向量空间上的线性变换。整个 FFT 过程可以理解为在不同二维子空间上反复施加此类线性映射，从而完成高维离散傅里叶变换。这种将高维线性变换拆解为若干低维线性变换的思想，与线性代数中通过分块矩阵简化运算的思想是一致的。

3.8 FFT 在图像压缩中的意义

在图像压缩问题中，FFT 及其相关变换主要体现了以下线性代数思想：首先，通过基变换将图像从像素基转换到频率基；其次，利用变换后系数的能量集中性，对幅值较小的高频分量进行舍弃；最后，依赖傅里叶变换的可逆性，实现压缩图像的近似重构。这些性质使 FFT 成为图像压缩中重要的理论工具。

4 PAC 数学原理

PAC (Principal Component Analysis of Channels, 通道主成分分析) 算法是线性代数理论在图像压缩领域的典型应用。尽管其在应用层面常被视为一种基于统计特性的特征提取技术，但其数学内核植根于矩阵的低秩近似理论。本节将从统计学与代数学两个维度，严谨阐述该算法的数学原理。

4.1 统计学视角：协方差与最大方差理论

在数学建模中，我们将图像的单颜色通道视为一个矩阵 $A \in \mathbb{R}^{H \times W}$ ，其中 H 为图像高度， W 为宽度。为了利用主成分分析 (PCA) 提取图像的主要特征，首先需要将图像的每一行视为 W 维空间中的一个数据样本。

设图像数据的行向量集合为 $\{\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_H\}$ ，其中 $\mathbf{a}_i \in \mathbb{R}^W$ 。为了消除原点偏移对二阶统计量的影响，必须对数据进行中心化处理。计算样本均值向量 μ ：

$$\mu = \frac{1}{H} \sum_{i=1}^H \mathbf{a}_i \quad (1)$$

中心化后的数据矩阵 X 定义为 $X = A - \mathbf{1}\mu$ ，其中 $\mathbf{1}$ 为 H 维全 1 列向量。此时，样本的协方差矩阵 C 可表示为：

$$C = \frac{1}{H-1} X^T X \quad (2)$$

协方差矩阵 C 是一个 $W \times W$ 的实对称矩阵。根据谱定理 (Spectral Theorem)，实对称矩阵必然可以被正交对角化，即存在正交矩阵 P 和对角矩阵 Λ ，使得 $C = P\Lambda P^T$ 。

PAC 算法的目标是寻找一组正交基，使得数据在该基上的投影方差最大。数学推导表明，这组最优基恰好是协方差矩阵 C 的特征向量（即矩阵 P 的列向量）。在图像信号中，绝大多数能量（方差）集中在前 k 个最大特征值对应的方向上，这为通过舍弃次要分量实现数据压缩提供了坚实的统计学基础。

4.2 代数视角：奇异值分解 (SVD)

在实际的数值计算中，显式构建协方差矩阵 $X^T X$ 不仅计算复杂度高达 $O(W^2 H)$ ，且极易引入浮点数截断误差，导致数值不稳定。因此，现代图像压缩算法通常直接对中心化矩阵 X 进行奇异值分解 (Singular Value Decomposition, SVD)。

对于任意实矩阵 $X \in \mathbb{R}^{H \times W}$ ，存在分解：

$$X = U \Sigma V^T \quad (3)$$

其中：

- $U \in \mathbb{R}^{H \times H}$ 是左奇异向量矩阵，其列向量构成了行空间的正交基；
- $\Sigma \in \mathbb{R}^{H \times W}$ 是对角矩阵，其对角元素 σ_i 为奇异值，且满足 $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ ；
- $V \in \mathbb{R}^{W \times W}$ 是右奇异向量矩阵，其列向量构成了定义域的正交基。

SVD 与 PCA 的内在联系可以通过以下推导揭示：

$$C \propto X^T X = (V \Sigma^T U^T)(U \Sigma V^T) = V(\Sigma^T \Sigma)V^T = V \Sigma^2 V^T \quad (4)$$

由此可见，SVD 分解得到的右奇异向量矩阵 V 恰好是协方差矩阵的特征向量矩阵，而奇异值 σ_i 与特征值 λ_i 满足关系 $\lambda_i \propto \sigma_i^2$ 。因此，SVD 提供了一种数值上更稳定、计算上更直接的方法来求解主成分。

4.3 最优低秩近似与 Eckart-Young-Mirsky 定理

PAC 压缩的核心步骤是矩阵截断（Truncation）。我们仅保留前 k 个最大的奇异值及其对应的奇异向量，构造近似矩阵 X_k ：

$$X_k = U_k \Sigma_k V_k^T \quad (5)$$

其中 U_k 为 U 的前 k 列， Σ_k 为前 k 个奇异值构成的对角阵， V_k^T 为 V^T 的前 k 行。

根据 **Eckart-Young-Mirsky 定理**，在所有秩为 k 的矩阵集合中， X_k 是原矩阵 X 在弗罗贝尼乌斯范数（Frobenius Norm）意义下的全局最优近似：

$$\min_{\text{rank}(Y)=k} \|X - Y\|_F = \|X - X_k\|_F = \sqrt{\sum_{i=k+1}^{\min(H,W)} \sigma_i^2} \quad (6)$$

该定理从数学上严格保证了，在给定的存储预算（即秩 k ）下，截断 SVD 能够保留原始图像最多的能量信息，是理论上的最优线性压缩方案。

5 PAC 实验分析

5.1 算法实现流程

为了验证上述理论的有效性，本文基于 Python 设计并实现了 PAC 图像压缩算法。算法流程严格遵循数学推导，针对 RGB 彩色图像的三个通道分别并行处理，具体步骤

如下：

1. **通道分离 (Channel Separation)**: 将输入的三维图像张量 $I \in \mathbb{R}^{H \times W \times 3}$ 分解为 R、G、B 三个独立的二维矩阵。
2. **中心化预处理 (Centering)**: 对每个通道矩阵计算行均值向量，并执行减均值操作。这一步至关重要，它确保了数据分布中心位于零点，从而使 SVD 分解能够正确反映数据的方差结构。
3. **截断编码 (Truncated Encoding)**: 利用 SVD 算法分解各通道矩阵，并仅保留前 k 个奇异分量 (U_k, Σ_k, V_k) 以及均值向量作为压缩载荷 (Payload)。
4. **逆向重建 (Reconstruction)**: 解码端通过矩阵乘法 $\hat{X} = U_k \Sigma_k V_k^T$ 恢复近似数据，加上均值向量后，将数值截断至 $[0, 255]$ 区间并合并通道，生成最终的重建图像。

5.2 实验设置

实验采用 Kodak Lossless True Color Image Suite 数据集，该数据集包含多幅纹理复杂度各异的高分辨率彩色图像（如 ‘kodim01.png’ 至 ‘kodim24.png’）。我们设定主成分保留数 $k \in \{5, 10, 20, 40, 80\}$ ，并采用以下指标进行量化评估：

- **PSNR (峰值信噪比)**: 衡量图像重建质量的客观标准，单位 dB，数值越高代表失真越小。
- **SSIM (结构相似性)**: 反映人眼对图像亮度、对比度和结构信息的感知程度，范围 $[0, 1]$ ，接近 1 表示结构保留完美。
- **CR (压缩率)**: 定义为原始图像大小与压缩后参数（采用 Float32 格式存储）大小的比值。

5.3 结果与讨论

为了深入分析算法在不同纹理特征下的表现，我们选取了纹理细节丰富的样本（Image 1）和结构相对简单的样本（Image 10）作为典型代表。详细实验数据如表 ?? 所示。

5.3.1 质量与秩的权衡分析

实验数据清晰地展示了压缩质量与秩 k 之间的正相关关系：

- **低秩区间 ($k = 5$)**: 此时压缩率极高 ($CR > 13$)，但 SSIM 较低 (Image 1 仅为 0.429)。图像丢失了大量高频纹理细节，仅保留了大致的颜色分布和低频轮廓，视觉体验较差。
- **中秩区间 ($k = 20 \sim 40$)**: 这是性能平衡的最优区间。PSNR 提升至 25dB 以上，SSIM 接近 0.7-0.8。算法在保持 2-4 倍压缩率的同时，成功恢复了图像的主要结构信息，达到了较好的视觉可用性。

表 1: 不同保留秩 k 下的图像压缩性能数据（数据来源：实验结果）

样本 ID	Rank (k)	MSE	PSNR (dB)	SSIM	CR (理论)
5*Image 1	5	495.72	21.18	0.429	13.70
	10	379.83	22.33	0.509	7.24
	20	282.66	23.62	0.593	3.73
	40	195.52	25.22	0.687	1.89
	80	116.29	27.48	0.789	0.95
5*Image 10	5	348.65	22.71	0.630	14.21
	10	225.15	24.61	0.663	7.38
	20	145.48	26.50	0.709	3.76
	40	84.70	28.85	0.779	1.90
	80	43.05	31.79	0.853	0.95

- **高秩区间 ($k = 80$):** 图像质量进一步提升 (PSNR > 27dB)，细节还原度极高，但存储代价显著增加。

5.3.2 存储精度与“负压缩”现象的工程分析

通过深入分析实验数据，我们发现了一个值得注意的工程现象：当 $k = 80$ 时，理论压缩率 CR 降至 0.95，出现了“负压缩”（即压缩后的文件反而比原图大）。这一现象的根本原因在于数据存储类型的差异。原始图像像素使用 8 位无符号整数（uint8，1 字节/像素）存储；而 PAC 算法分解出的奇异向量矩阵（ U 和 V ）是连续实数，通常需要 32 位浮点数（float32，4 字节/数值）以维持运算精度。虽然 SVD 大幅减少了数据的“维度数量”（从 $H \times W$ 降维到 k 维子空间），但单个数据的“位宽”增加了 4 倍。当保留的秩 k 较大时，维度缩减带来的红利被浮点数存储的开销完全抵消。这表明，基于 SVD 的压缩方法如果直接存储浮点矩阵，并不适合高保真的无损级压缩。在实际工程应用中，必须结合量化（Quantization）技术将浮点数降级（如转为 float16 或 uint8），才能在高 k 值下重获压缩优势。

5.3.3 图像内容的频谱敏感性

对比 Image 1（复杂纹理）和 Image 10（简单结构）的数据可以发现：在相同的 $k = 40$ 下，Image 10 的 PSNR (28.85 dB) 显著优于 Image 1 (25.22 dB)。这验证了 PAC 算法对图像内容的频谱分布具有高度敏感性。Image 10 包含更多平滑区域，其能量在频域上更集中于少数几个最大的奇异值（奇异值衰减速度更快），因此该算法能以更低的信息代价（更小的 k ）还原更多的图像信息。这一特性也体现了 SVD 作为一种“自适应基变换”，相较于固定基变换（如 FFT/DCT）更能捕捉数据的内在统计结构。

参考文献

- [1] A. V. Oppenheim, R. W. Schaffer, *Discrete-Time Signal Processing*, Prentice Hall, 2010.